Architecture et Bonnes Pratiques pour une SPA Complète

Objectifs

1. Structurer un projet React robuste, maintenable et évolutif.
2. Comprendre comment imbriquer Redux Toolkit, React Router et les optimisations au sein d'une architecture globale.
3. Isoler la logique métier de la logique d'affichage grâce aux Hooks personnalisés (Custom Hooks).
4. S'approprier le cahier des charges et la méthodologie de développement pour le projet final.

1. Structuration et Arborescence des Fichiers

Lorsque l'on développe une Single Page Application complète, une application initialisée par Vite commence avec un dossier `src` quasiment vide. Il est crucial d'adopter une convention de nommage et de rangement des fichiers claire pour que l'équipe puisse collaborer efficacement.

Il n'y a pas d'architecture "officielle" imposée par React, mais l'industrie a convergé vers des standards très répandus.

1.1 L'Architecture Classique par Nature (Type-based)

C'est l'approche la plus simple et la plus utilisée pour les projets de taille petite à moyenne. On regroupe les fichiers selon leur nature technique.

```
None
src/
├── api/             // Fonctions pour les appels réseaux (fetch, axios)
├── assets/          // Images, polices, fichiers statiques
├── components/      // Composants réutilisables, partagés (Boutons, Cartes,
Modales)
├── hooks/           // Hooks personnalisés (useAuth, useFetch)
├── pages/           // Composants de vues principales (associées au Router)
├── store/           // Configuration Redux (store.js) et les Slices
├── utils/           // Fonctions utilitaires pures (formatage de dates, calculs)
├── App.jsx          // Point d'entrée du routage
└── main.jsx         // Montage React, Providers (Redux, Router)
```

1.2 Distinguer "Pages" (Vues) et "Components"

- **Pages (ou Views) :** Ce sont les composants racines de chaque route (ex: `DashboardPage`, `StationDetailsPage`). Ils ont souvent la responsabilité d'interroger le Store Redux ou l'API, et d'orchestrer la mise en page globale.
- **Components :** Ce sont des briques d'interface réutilisables et plus "naïves" (ex: `FilterBar`, `StationCard`). Ils reçoivent idéalement leurs données via des `props` et renvoient des événements via des fonctions de rappel (callbacks).

2. Imbrication des Outils : Le Cœur de l'Application

Le succès d'une SPA repose sur l'assemblage correct des différentes bibliothèques au sommet de l'arbre des composants (dans `main.jsx` ou `App.jsx`).

2.1 Ordre des Providers

L'application doit être enveloppée par différents contextes globaux. L'ordre est important : le Router doit souvent être à l'intérieur du Provider Redux si des composants liés au routage ont besoin de lire le Store, ou inversement selon les cas d'usage avancés.

L'architecture standard ressemble à ceci :

JavaScript

```
// src/main.jsx
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import { Provider } from 'react-redux';
import { BrowserRouter } from 'react-router-dom';
import { store } from './store/store';
import App from './App';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <Provider store={store}>
      <BrowserRouter>
        <App />
      </BrowserRouter>
    </Provider>
  </StrictMode>
);
```

2.2 Orchestration dans App.jsx

Le composant `App.jsx` ne doit pas contenir de logique métier complexe ni de rendu visuel lourd. Il sert généralement de "Table de Routage" et définit la structure de base (gabarit).

JavaScript

```
// src/App.jsx
import { Routes, Route } from 'react-router-dom';
import GlobalLayout from './components/GlobalLayout';
import DashboardPage from './pages/DashboardPage';
import SettingsPage from './pages/SettingsPage';

export default function App() {
  return (
    <Routes>
      {/* GlobalLayout contiendra le Header et un <Outlet /> */}
      <Route path="/" element={<GlobalLayout />}>
        <Route index element={<DashboardPage />} />
        <Route path="settings" element={<SettingsPage />} />
      </Route>
    </Routes>
  );
}
```

3. Isoler la Logique : Les Hooks Personnalisés (Custom Hooks)

L'un des problèmes fréquents dans les gros composants React est l'accumulation de Hooks (`useState`, `useEffect`) qui polluent la lecture du JSX.

React permet de créer ses propres Hooks (des fonctions JavaScript dont le nom commence par `use`) pour extraire la logique métier.

Exemple : Séparer l'API de l'Affichage

Au lieu d'écrire toute la logique d'appel réseau dans la vue :

JavaScript

```
// src/hooks/useStationData.js
import { useState, useEffect } from 'react';

export function useStationData(stationId) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
```

```
  useEffect(() => {
    let isMounted = true;

    fetch(`/api/stations/${stationId}`)
      .then(res => res.json())
      .then(result => {
        if (isMounted) {
          setData(result);
          setLoading(false);
        }
      });

    return () => { isMounted = false; };
  }, [stationId]);

  return { data, loading };
}
```

Le composant devient ainsi purement déclaratif :

JavaScript

```
// src/pages/StationDetailsPage.jsx
import { useParams } from 'react-router-dom';
import { useStationData } from '../hooks/useStationData';

export default function StationDetailsPage() {
  const { id } = useParams();
  const { data, loading } = useStationData(id);

  if (loading) return <Spinner />;

  return <div><h1>{data.name}</h1></div>;
}
```

4. Préparation au Projet Final : Méthodologie

Le projet final consiste à créer une application complète en partant de zéro. Voici les étapes méthodologiques recommandées :

1. **Découpage de l'interface (Wireframing / Mockup) :** Identifiez les grandes zones visuelles (En-tête, Sidebar, Contenu principal).

2. **Définition des Routes :** Listez les URL de votre application (`/`, `/stations`, `/stations/:id`).
3. **Architecture de l'État Global :** Que doit contenir Redux ? (Ex: la liste des stations, les filtres actifs, le thème de l'application).
4. **Développement des briques de base :** Créez d'abord les petits composants statiques (Boutons, Cartes).
5. **Connexion et Interactivité :** Intégrez Redux, les appels asynchrones (`createAsyncThunk`) et la navigation.
6. **Optimisation finale :** Appliquez `React.memo` ou `useMemo` uniquement si vous observez des ralentissements lors des tests.

5. Synthèse des Bonnes Pratiques

- **DRY (Don't Repeat Yourself) :** Si un morceau d'interface est copié-collé deux fois, il doit devenir un composant. Si une logique d'état est copiée-collée, elle doit devenir un Custom Hook.
- **Responsabilité Unique :** Un fichier = un composant. Un composant fait une seule chose bien.
- **Éviter le "Prop Drilling" :** Si une donnée traverse plus de 3 niveaux d'enfants sans être utilisée par les intermédiaires, utilisez Redux.
- **L'Optimisation vient à la fin :** Concentrez-vous d'abord sur un code fonctionnel, clair et lisible. L'optimisation prématurée est la racine de nombreux bugs.